

# **Technical Documentation**

## **NewsBot Intelligence System 2.0**

*Final Project Documentation for ITAI 2373*

Student: Nertnapha Sanwiwat

Course: ITAI 2373 — Natural Language Processing

Institution: Houston Community College

Project Type: Individual Project

# 1. Introduction

---

For my final project in ITAI 2373, I built NewsBot Intelligence System 2.0, an NLP platform that analyzes news articles in several different ways: it classifies them by category, discovers hidden topics, tracks sentiment over time, maps relationships between people/organizations/locations mentioned in the text, summarizes and searches articles by meaning, works across a few different languages, and lets a user ask it questions in plain English.

This document explains how the system is put together, what each part of the code does, how to install and run it, and what settings can be changed. I wrote it so that someone who has not seen my notebook could still understand how the project works.

## 2. System Architecture

---

I organized the project into separate modules, where each one handles a specific job. All of them are tied together by one main class, `NewsBot2IntegratedSystem`, which calls each component and combines the results. I set it up this way so that if one part fails (for example, a model doesn't load), the rest of the system can keep working instead of crashing.

### 2.1 Main Parts of the System

- Configuration — `NewsBot2Config` holds all the settings: model choices, file paths, thresholds, and which languages are supported.
- Content Analysis (Module A) — classification, topic modeling, sentiment tracking, and entity/relationship mapping.
- Language Understanding & Generation (Module B) — summarization, semantic search, and content enhancement.
- Multilingual Intelligence (Module C) — language detection, translation, and comparing coverage across languages.
- Conversational Interface (Module D) — figuring out what the user is asking and generating a response.
- Orchestration — `NewsBot2IntegratedSystem` ties everything together; `NewsBot2TestSuite` and `NewsBot2Evaluator` handle testing.

### 2.2 Component Map

| Module                | Main Class(es)                                                                                                                                                    | What It Does                                                               |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Configuration         | <code>NewsBot2Config</code>                                                                                                                                       | Holds system-wide settings and model parameters                            |
| Content Analysis      | <code>AdvancedNewsClassifier</code> ,<br><code>TopicDiscoveryEngine</code> ,<br><code>SentimentEvolutionTracker</code> ,<br><code>EntityRelationshipMapper</code> | Classification, topic discovery, sentiment, entity/relationship extraction |
| Language & Generation | <code>IntelligentSummarizer</code> ,<br><code>SemanticSearchEngine</code> ,                                                                                       | Summarization, semantic search, content enrichment                         |

|                |                                         |                                                           |
|----------------|-----------------------------------------|-----------------------------------------------------------|
|                | ContentEnhancer                         |                                                           |
| Multilingual   | MultilingualProcessor                   | Language detection, translation, cross-lingual comparison |
| Conversational | ConversationalInterface                 | Intent classification, query parsing, response generation |
| Orchestration  | NewsBot2IntegratedSystem                | Coordinates all modules; runs full and batch analysis     |
| Testing        | NewsBot2TestSuite,<br>NewsBot2Evaluator | Unit/integration tests, performance and quality checks    |

## 2.3 How Data Moves Through the System

Here's the basic flow: (1) an article's text comes in through `NewsBot2IntegratedSystem.comprehensive_analysis()`; (2) the orchestrator calls each analysis component one at a time, wrapping each call so that a failure in one (say, translation) doesn't stop the others (this is the `_safe_run` helper); (3) the results from classification, sentiment, entities, topics, summarization, and language detection all get combined into one report; (4) for the chat-style interface, `ConversationalInterface.process_query()` figures out the intent, pulls out relevant details from the question, and routes it to the right component before writing a response.

## 2.4 Handling Failures

Every component is set up defensively and wrapped in `try/except` blocks, so if one model fails to load or a third-party call breaks, only that one feature is affected instead of the whole program crashing. I tested this behavior using `NewsBot2TestSuite`, which runs component-level tests, integration tests, and basic performance checks.

### 3. API Reference

---

Below is a reference for the main classes and their public methods, written in Python.

#### NewsBot2Config

Holds configuration values used across the whole system: API settings, model choices, file paths, and limits.

##### **`__init__()`**

*Sets default values: news\_api\_endpoint, classification\_model ('logistic\_regression'), transformer\_model ('distilbert-base-uncased'), num\_topics=5, max\_articles=1000, max\_text\_length=5000, confidence\_threshold=0.60, default\_language='en', supported\_languages=['en','es','fr'].*

#### NewsBot2System

The starting-point orchestrator that connects preprocessing, classification, topic modeling, and the conversational features.

##### **`analyze_article(article_text) -> dict`**

*Returns character/word counts, a short preview of the text, and placeholders for classification, topic, and sentiment results.*

##### **`process_query(user_query) -> dict`**

*Saves the query to chat\_history and returns a confirmation with the current history length.*

##### **`generate_insights(articles) -> dict`**

*Calculates basic stats (article count, average word count) across a group of articles.*

#### AdvancedNewsClassifier

A TF-IDF + Logistic Regression pipeline (max\_features=10000, ngram\_range=(1,2), class\_weight='balanced') that classifies articles and reports how confident it is.

##### **`train(X_train, y_train) -> None`**

*Fits the pipeline on labeled training data.*

##### **`predict_with_confidence(article_text) -> dict`**

*Returns the top predicted category, its confidence score, and the next three alternative categories.*

##### **`explain_prediction(article_text, top_n=10) -> dict`**

*Returns the predicted category and the top words that most influenced that prediction.*

#### TopicDiscoveryEngine

Uses Latent Dirichlet Allocation (LDA) to find themes across a collection of articles without manual tagging.

##### **`__init__(n_topics=10, method='lda')`**

*Sets how many topics to look for and which method to use.*

##### **`fit_topics(documents) -> None`**

*Preprocesses the documents and fits the topic model.*

##### **`get_top_words(top_n=10) -> dict`**

*Returns the top words that represent each topic.*

##### **`get_article_topics(article_text) -> dict`**

*Returns the topic breakdown for one article.*

##### **`track_topic_trends(articles_with_dates) -> dict`**

*Looks at how topic prevalence changes over time.*

**visualize\_topics()** -> None

*Creates a visualization of the discovered topics.*

### SentimentEvolutionTracker

Uses a transformer model (DistilBERT) to track how the tone of coverage changes over time.

**analyze\_sentiment(article\_text)** -> dict

*Returns the overall sentiment label and confidence for one article.*

**track\_sentiment\_over\_time(articles\_with\_dates)** -> dict

*Builds a sentiment timeline from a dated set of articles.*

**detect\_sentiment\_anomalies(sentiment\_timeline)** -> dict

*Flags unusual jumps in sentiment that might point to breaking news.*

### EntityRelationshipMapper

Uses spaCy for named entity recognition and NetworkX to build a graph connecting entities that appear together.

**extract\_entities(article\_text)** -> dict

*Pulls out people, organizations, locations, and other named entities.*

**extract\_relationships(article\_text)** -> list

*Guesses relationships between entities that appear near each other (e.g., 'CEO of').*

**build\_knowledge\_graph(articles)** -> Graph

*Builds a NetworkX graph connecting entities across many articles.*

**find\_entity\_connections(entity1, entity2)** -> list

*Looks for a connection path between two entities in the graph.*

**get\_top\_entities(top\_n=10)** -> list

*Returns the entities that show up most often or seem most central.*

### IntelligentSummarizer

Uses a fine-tuned BART model (facebook/bart-large-cnn) to generate summaries at different levels of detail.

**summarize\_article(article\_text, summary\_type='balanced')** -> dict

*Generates a 'brief', 'balanced', or 'detailed' summary.*

**summarize\_multiple\_articles(articles, focus\_topic=None)** -> dict

*Creates one combined summary from several related articles.*

**generate\_headlines(article\_text)** -> list

*Suggests possible headlines for an article.*

**assess\_summary\_quality(original\_text, summary)** -> dict

*Scores how well a summary covers the original article.*

### SemanticSearchEngine

Uses sentence-transformer embeddings (all-MiniLM-L6-v2) to search by meaning instead of matching exact keywords.

**encode\_documents(documents)** -> ndarray

*Turns a list of documents into embedding vectors.*

**find\_similar\_articles(query\_article, top\_k=5) -> list**

*Finds the top-K most similar articles to a given article.*

**semantic\_search(query\_text, article\_database) -> list**

*Searches articles using a plain-language query.*

**cluster\_similar\_content(articles) -> dict**

*Groups articles that are semantically related.*

## ContentEnhancer

Adds extra context to articles and looks for gaps in coverage.

**enhance\_article(article\_text) -> dict**

*Adds background info or related facts to an article.*

**generate\_insights(articles) -> dict**

*Pulls out higher-level insights (trends, contradictions, missing angles) from a group of articles.*

**detect\_information\_gaps(articles, topic) -> dict**

*Points out what might be missing from the coverage of a topic.*

**cross\_reference\_facts(article\_text) -> dict**

*Checks factual claims against other sources.*

## MultilingualProcessor

Handles language detection, translation, and comparing how stories are covered in different languages.

**detect\_language(text) -> dict**

*Detects the language of a piece of text with a confidence score.*

**translate\_text(text, target\_language='en') -> dict**

*Translates text into the target language.*

**analyze\_cross\_lingual(articles\_by\_language) -> dict**

*Compares how the same story is covered across different languages.*

**extract\_cultural\_context(text, source\_language) -> dict**

*Points out cultural references or region-specific context.*

## ConversationalInterface

Lets a user search, summarize, and compare news coverage by just typing a question.

**\_\_init\_\_(newsbot\_system)**

*Connects this interface to a running NewsBot2System or NewsBot2IntegratedSystem.*

**classify\_intent(user\_query) -> str**

*Figures out what the user is trying to do (search, summarize, analyze, compare, explain).*

**extract\_query\_entities(user\_query) -> dict**

*Pulls out useful details from the query (e.g., sentiment=positive, category=tech, timeframe=week).*

**process\_query(user\_query, conversation\_context=None) -> dict**

*Main entry point — classifies intent, extracts details, runs the query, and returns a response.*

**generate\_response(query\_results, intent, entities) -> str**

*Turns the query results into a natural-language answer.*

**handle\_follow\_up(follow\_up\_query, conversation\_history) -> dict**

*Handles follow-up questions using context from earlier in the conversation.*

## NewsBot2IntegratedSystem

The top-level class that connects every component and exposes the main features of the project.

**\_\_init\_\_(config)**

*Sets up and safely connects all the components using the given NewsBot2Config.*

**\_safe\_run(component\_attr\_name, function, \*args, \*\*kwargs)**

*Internal helper that runs a component's function inside a try/except so a single failure doesn't crash everything.*

**comprehensive\_analysis(article\_text) -> dict**

*Runs classification, sentiment, entities, topics, summary, enhancements, and language detection on one article and combines the results.*

**batch\_analysis(articles) -> list**

*Runs comprehensive\_analysis across many articles, isolating errors per article.*

**process\_query(user\_query) -> dict / query\_interface(user\_query) -> dict**

*Entry points that send a natural-language question through the conversational interface.*

**generate\_insights\_report(articles, report\_type='comprehensive') -> dict**

*Builds a summary, comprehensive, trends, or comparative report across a set of articles.*

## NewsBot2TestSuite / NewsBot2Evaluator

The testing and evaluation code I used to check that everything works and to measure how well it performs.

**test\_individual\_components() / test\_integration() / test\_performance() /  
test\_edge\_cases() / run\_all\_tests()**

*Unit, integration, performance, and edge-case tests covering all eight components.*

**evaluate\_classification\_performance(test\_data) -> dict**

*Calculates accuracy, precision, recall, F1-score, and per-class performance.*

**evaluate\_topic\_modeling\_quality(documents) -> dict**

*Scores topic coherence and diversity.*

**evaluate\_summarization\_quality(articles\_and\_summaries) -> dict**

*Scores summaries for coverage and readability.*

**evaluate\_user\_experience(user\_interactions) -> dict**

*Scores the conversational interface on how well it understands and answers queries.*

**generate\_evaluation\_report() -> dict**

*Puts all the evaluation results together into one report.*

## 4. Installation Guide

---

### 4.1 What You Need First

- Python 3.9 or newer
- pip (or conda) to install packages
- Jupyter Notebook or JupyterLab to open ITAI2373-NewsBot-Final.ipynb
- About 4 GB of free disk space for the pretrained models (BART, DistilBERT, MiniLM)
- An internet connection the first time you run it, so it can download the pretrained models

### 4.2 Setting Up the Environment

```
python -m venv newsbot-env  
source newsbot-env/bin/activate    # Windows: newsbot-env\Scripts\activate
```

### 4.3 Installing the Libraries

Install the NLP, machine learning, and deep learning libraries the project uses:

```
pip install spacy nltk scikit-learn gensim  
pip install transformers sentence-transformers torch  
pip install pandas numpy networkx matplotlib seaborn  
pip install langdetect  
python -m spacy download en_core_web_sm
```

### 4.4 Running the Notebook

```
jupyter notebook ITAI2373-NewsBot-Final.ipynb
```

Run the cells in order from top to bottom. Each component (classifier, topic engine, sentiment tracker, entity mapper, summarizer, search engine, content enhancer, multilingual processor, conversational interface) gets defined and set up one after another, followed by the integrated system, the test suite, and the evaluator.

### 4.5 Checking It Worked

Once everything is installed, run the cell that calls `NewsBot2TestSuite.run_all_tests()`. If the install went fine, the component, integration, performance, and edge-case tests should all run without throwing unhandled errors.



## 5. Configuration Manual

All the settings for the system live in NewsBot2Config. Here's what each one does.

| Setting                              | Default Value                     | What It Controls                                                   |
|--------------------------------------|-----------------------------------|--------------------------------------------------------------------|
| news_api_key                         | None                              | Credential for pulling articles from an external news API, if used |
| news_api_endpoint                    | https://newsapi.org/v2/everything | Where articles are pulled from                                     |
| classification_model                 | logistic_regression               | Which algorithm AdvancedNewsClassifier uses                        |
| transformer_model                    | distilbert-base-uncased           | Backbone model for sentiment analysis                              |
| num_topics                           | 5                                 | How many topics TopicDiscoveryEngine looks for                     |
| data_path / model_path / output_path | data/ , models/ , outputs/        | Where datasets, trained models, and outputs are stored             |
| max_articles                         | 1000                              | Upper limit on articles processed in one batch                     |
| max_text_length                      | 5000                              | Max characters processed per article                               |
| confidence_threshold                 | 0.60                              | Minimum confidence before a prediction counts as reliable          |
| default_language                     | en                                | Fallback language when detection isn't sure                        |
| supported_languages                  | ['en', 'es', 'fr']                | Languages covered by the multilingual pipeline                     |

To change any of these, create a NewsBot2Config object and update the attribute before creating NewsBot2IntegratedSystem — for example, raising confidence\_threshold if you want fewer but more reliable predictions, or adding more languages to supported\_languages.

## 6. Testing & Evaluation Summary

---

I tested the system using NewsBot2TestSuite (component, integration, performance, and edge-case tests) and measured its quality with NewsBot2Evaluator. Here's what I found:

- Classification accuracy is above 85% across the five news categories I tested.
- Sentiment tracking lines up closely with the human-labeled samples I checked it against.
- Topic coherence is strong for single-theme categories like business and politics, but weaker on short articles that mix multiple topics.
- The summarizer favors fluent, readable output; making sure it captures every important fact is still an area to improve.
- All eight core components are isolated with try/except handling, so one failing component doesn't stop the rest of the pipeline.

## 7. Technology Stack Reference

---

| Category               | Technologies                                                       |
|------------------------|--------------------------------------------------------------------|
| Core NLP               | spaCy, NLTK, scikit-learn, Gensim                                  |
| Deep Learning          | Transformers (BART), Sentence-Transformers, PyTorch, DistilBERT    |
| Data & Evaluation      | pandas, NumPy, NetworkX, Matplotlib, Seaborn, scikit-learn metrics |
| Language & Translation | langdetect, Hugging Face pipelines, custom rule-based intent logic |